

MLFF-DATA9A: Integration and Production Qualification

mdstats project

2026-07-28

MLFF-DATA9A: integration and production qualification

Purpose

DATA9A is the hard gate between deterministic dataset preparation and expensive MACE training. It corrects and verifies every assumption that depends on the actual foundation checkpoint, replay files, installed MACE runtime, and production LTA dataset. DATA9A does not select a final checkpoint and does not activate locked tests.

Required corrections

Foundation-residual atomic references

Training from scratch may fit

$$A\mathbf{e}_0 \approx \mathbf{E}_{\text{DFT}}.$$

Foundation-model fine-tuning instead fits checkpoint-bound corrections

$$A\Delta\mathbf{e}_0 \approx \mathbf{E}_{\text{DFT}} - \mathbf{E}_{\text{foundation}},$$

then exports

$$\mathbf{e}_0^{\text{target}} = \mathbf{e}_0^{\text{foundation}} + \Delta\mathbf{e}_0.$$

The fit is local to each final or cross-validation training domain. Rank, singular values, null-space dimension, residuals, foundation predictions, foundation E0 values, corrections, final E0 values, and checkpoint SHA-256 are all immutable record fields. A from-scratch fit is rejected by the production foundation adapter unless an explicit diagnostic override is recorded.

Portable MACE artifacts

The default bundle stages the exact foundation checkpoint under **shared/foundation/** and replay files under **shared/replay/**. All YAML paths are relative to the job directory. Each run script resolves its own directory and changes into it before invoking **mace_run_train**. Moving the complete bundle therefore preserves its path contract.

Complete extended-XYZ verification

The exporter rejects nonfinite cells, positions, energies, forces, stresses, or weights. The ASE write/read round trip verifies configuration count, frame UID, config type, species and order, PBC, cell, positions, energy, forces, stress, and all configuration/property weights.

Replay qualification

Every replay file must provide finite energies and correctly shaped finite forces. Stress coverage is recorded rather than assumed. Exact duplicate geometries within a replay file are rejected. Replay train and monitor files must be disjoint. Pseudo-label replay records the exact foundation checkpoint digest used to generate labels. The MACE atomic-number set is the union of target and replay elements. Target/replay exact-geometry overlap is rejected by default.

Selection-level identity

DATA8/DATA9A accepts an explicit DATA7 ladder size. The chosen size is part of `TrainingProtocolIdentity`. Learning-curve job families must therefore be created as distinct signed protocols rather than by editing one YAML file.

Installed MACE qualification

`InstalledMaceQualificationRecord` binds:

- MACE source-tree digest and version;
- Python executable and version;
- PyTorch and ASE versions;
- compile result;
- top-level MACE import result;
- `mace.cli.run_train` import result;
- required and optional dependency availability;
- exact blocking exception when qualification is incomplete.

Compatibility stubs are forbidden. A source-only qualification does not permit training execution.

The initial lock is `mace-torch==0.3.16`. Before DATA9B, the environment must also pass a naive one-configuration dry run, a two-head replay dry run, a short CPU smoke training, checkpoint enumeration, target-head extraction, and `mace_eval_configs` round trip.

Offline runtime bootstrap

DATA9A reads the complete MACE dependency contract directly from the supplied `setup.cfg`. A dependency record contains the normalized distribution name, Python import name, version specifier, requirement text, and the SHA-256 of the source dependency declaration. This avoids drift between mdstats and MACE.

`create_mace_runtime_environment()` accepts only explicit artifacts:

- the MACE source archive;
- the ASE source archive;
- optional build-tool wheels;
- a local dependency wheelhouse.

Offline mode is the default. Pip is invoked with `--no-index`, source archives are installed with `--no-deps`, and each artifact and command is recorded. The base scientific environment may be inherited explicitly for large packages such as PyTorch, but inherited paths are written into the environment and remain part of the runtime provenance.

`MaceRuntimeEnvironmentRecord` reports every required distribution and import, its observed version, installation command results, MACE/ASE/PyTorch versions, the base interpreter, every inherited base-Python search path, declared-version specifier mismatches, and the first blocking import exception. A successful import is insufficient when the observed distribution version violates the MACE `setup.cfg` requirement. `run_mace_cli_smoke()` returns an immutable `MaceCliSmokeRecord` and refuses to run commands until the environment is complete. The initial commands are `mace_run_train --help` and `mace_eval_configs --help`. DATA9A2 then binds each DATA8 job to a `MaceConfigRealizationRecord` and a bounded `MaceJobExecutionSmokeRecord`, covering the genuine parser, loader dry run, one-epoch CPU training, checkpoint inventory, head enumeration, target-head extraction, and `mace_eval_configs` round trip.

The helper command

```
python tools/qualify_mace_runtime.py \
  --environment mace-env \
  --mace-source mace-src \
  --mace-archive mace_torch-0.3.16.tar.gz \
  --ase-archive ase-3.29.0.tar.gz \
  --wheelhouse wheelhouse \
  --build-tool setuptools.whl \
  --output qualification.json \
  --recreate
```

creates a replayable qualification record. A missing package is not replaced by a stub.

Current supplied-package result

The supplied offline wheelhouse, ASE 3.29.0 source, MACE 0.3.16 source, and PyTorch 2.10.0+cpu base environment now pass complete dependency and CLI qualification. The supplied MACE-MPA-0 medium checkpoint loads successfully. The locked dependency evidence includes `e3nn==0.4.4` and its transitive `opt-einsum-fx>=0.1.4` requirement; the installer never inserts dependency stubs.

DATA9A2 identified and corrected the DATA8 runtime boundary: MACE v0.3.16 requires scalar Python-literal strings for `atomic_numbers`, `heads`, and nested head `E0s`; requires lowercase `universal`; and does not accept `weight_pt` or `weight_ft`. Fixed-file target/replay exposure is therefore realized in extended-XYZ `config_weight` values. Corrected native DATA8 jobs pass real naive and two-head replay dry runs, one-epoch CPU training, checkpoint enumeration, `pt_head/target_head` listing, target-head extraction, and finite energy/force/stress evaluation round trips.

DATA9A3 qualified the complete intended bulk-LTA target corpus through DATA5: 27 sources, 37,632 complete ionic frames, 97 partition units, 25 inferred conditions, three outer folds, and a passing leakage audit. The target-corpus qualification is therefore closed. The full DATA9A gate remains conditionally open only for downstream artifacts that require additional scientific inputs or production realization: completion of the exact frozen production-corpus plan, checkpoint-bound DATA6 descriptors/predictions, verified residual-E0 DATA7 fits, DATA8 generation evidence, and the exact production replay corpus used by DATA9B.

Production qualification

Production qualification is bound to `ProductionCorpusPlan`; caller-supplied counts or readiness Booleans cannot replace evidence. Foundation descriptors/predictions, residual-E0 fits, replay numerical labels, optional extension coverage, and DATA8 generation identity are derived from verified native artifacts.

Before cross-validation or final training, all intended LTA trajectories must be qualified through the available preparation stages. DATA9A3 has completed DATA2-DATA5 for all 27 sources and records source compatibility, rejected/degraded frames, role feasibility, independence grades, leakage status, runtime, peak memory, cache sizes, and artifact sizes. DATA6-DATA8 must then add site coverage, foundation descriptors, residual-E0 conditioning, selection ladders, and executable training artifacts. Missing required profile-extension evidence, calibration cohort, independent test, or replay corpus is reported, not fabricated.

Gate

DATA9A passes only when:

1. every foundation fine-tuning job uses a checkpoint-bound residual E0 fit;
2. MACE artifacts are relocatable and pass complete numeric round trips;
3. replay data pass property, provenance, element, and duplicate audits;
4. the requested training ladder size is explicit;
5. the real MACE runtime passes dependency-complete installed qualification, CLI help probes, naive and replay smoke training, checkpoint enumeration, head extraction, and evaluation round trip;
6. the real MPA-0 checkpoint and replay files are identified by SHA-256;
7. the complete production dataset passes preparation and resource benchmarks.

Deferred to DATA9B

DATA9B owns process execution at scale, independent cross-validation training, save-all checkpoint evaluation, target/focus-group/stress/replay constraints, learning curves, naive-versus-replay comparison, multiple seeds, final committee construction, target-head extraction, and protocol freeze.